

Module 8

UVM Miscellaneous Utilities

Learning objectives

- Master UVM utility classes and helper functions

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["UVM Command Line Processor ("]

T2["UVM Comparators"]

T1 --> T2

T3["UVM Recorders"]

T2 --> T3

Master UVM utility classes and helper functions

Overview

- This module covers the miscellaneous utilities provided by UVM that support verification environmen...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

CLP[CLP plusargs] --> CFG[test config]

CFG --> ENV[existing uvm_env]

subgraph util["UVM utilities"]

CMP[comparators]

REC[recorders]

Module 8: DUT structure and key interfaces (see common_dut/rtl/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

MON[monitor] --> AP[analysis port]

AP --> CMP[comparator]

AP --> REC[recorder]

AP --> SB[scoreboard]

Module 8: UVM layers, agents, and checkers for this phase.

1. Utility layer in the testbench

- No new large DUT focus — utilities plug into existing envs from prior modules
- CLP (``uvm_cmdline_processor``) configures tests without recompilation
- Comparators, recorders, pools, queues support analysis and debug infrastructure

2. How utilities attach

- Comparators sit on analysis ports beside scoreboards
- Recorders log transactions to files for offline review
- Pools/queues manage recycled objects and ordered work lists
- String/math/random helpers reduce boilerplate in checks and stimulus

3. Integration architecture

- ``module8/examples/integration/`` wires multiple utilities in one mini-env
- Tests under ``module8/tests/pyuvm_tests/`` prove utilities under simulation
- Pattern: configure via CLP → run sequence → compare → record → report

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

RUN[run test] --> CMP[compare stream]

CMP --> REC[record transactions]

REC --> RPT[report + CLP filters]

Module 8: how tests, checks, and metrics close verification goals.

1. Utility-focused test methods

- Comparator tests: equal/less/greater policies on transaction streams
- Recorder tests: verify log format and flush on phase boundaries
- CLP tests: plusargs override seed, verbosity, and test name

2. Efficiency and reuse

- Object pools cut allocation churn in high-transaction regressions
- Queues order deferred checks or secondary stimulus
- Random/string/math utils keep constraints readable in sequences

3. Closure

- ``./scripts/module8.sh --check`` across all utility examples
- Assessment: CLP, comparators, recorders, pools, queues, integration
- Capstone: combine utilities with Module 6/7 style envs in your own project

Syllabus topics

1. UVM Command Line Processor (CLP) (1/4)

- Command Line Processor Overview
- What is CLP?
- Why use CLP?
- CLP benefits
- CLP vs manual argument parsing
- CLP Usage

1. UVM Command Line Processor (CLP) (2/4)

- Getting command-line arguments
- Argument types (string, int, bit, time)
- Default values
- Argument validation
- CLP Methods
- ``get_arg_value()` - Get argument value

1. UVM Command Line Processor (CLP) (3/4)

- ``get_arg_values()` - Get multiple values
- ``get_arg_count()` - Get argument count
- ``has_arg()` - Check if argument exists
- CLP Patterns
- Test configuration via command line
- Debug control via command line

1. UVM Command Line Processor (CLP) (4/4)

- Simulation control via command line
- Best practices

2. UVM Comparators (1/5)

- Comparator Overview
- What are comparators?
- Comparator purpose
- When to use comparators
- Comparator types
- Built-in Comparators

2. UVM Comparators (2/5)

- ``uvm_in_order_comparator`` - In-order comparison
- ``uvm_algorithmic_comparator`` - Algorithmic comparison
- Comparator characteristics
- Comparator selection
- In-Order Comparator
- Sequential comparison

2. UVM Comparators (3/5)

- Transaction matching
- Comparison logic
- Error reporting
- Algorithmic Comparator
- Custom comparison algorithms
- Flexible matching

2. UVM Comparators (4/5)

- Comparison functions
- Use cases
- Comparator Implementation
- Creating comparators
- Connecting comparators
- Configuring comparators

2. UVM Comparators (5/5)

- Using comparators in scoreboards

3. UVM Recorders (1/5)

- Recorder Overview
- What are recorders?
- Recorder purpose
- Transaction recording
- Recording benefits
- Recorder Types

3. UVM Recorders (2/5)

- `uvm\_text\_recorder` - Text recording
- `uvm\_tr\_database` - Transaction database
- Recording formats
- Recording selection
- Recorder Usage
- Enabling recording

3. UVM Recorders (3/5)

- Recording transactions
- Recording configuration
- Recording analysis
- Transaction Recording
- Recording sequence items
- Recording transactions

3. UVM Recorders (4/5)

- Recording timing
- Recording relationships
- Recorder Implementation
- Creating recorders
- Connecting recorders
- Configuring recorders

3. UVM Recorders (5/5)

- Analyzing recordings

4. UVM Pools (1/5)

- Object Pool Overview
- What are pools?
- Pool purpose
- Object reuse
- Performance benefits
- Pool Types

4. UVM Pools (2/5)

- `uvm\_pool` - Generic object pool
- Pool characteristics
- Pool operations
- Pool use cases
- Pool Usage
- Creating pools

4. UVM Pools (3/5)

- Adding objects to pools
- Getting objects from pools
- Pool management
- Pool Implementation
- Pool creation
- Object allocation

4. UVM Pools (4/5)

- Object deallocation
- Pool cleanup
- Pool Patterns
- Transaction pooling
- Sequence item pooling
- Performance optimization

4. UVM Pools (5/5)

- Memory management

5. UVM Queues (1/5)

- Queue Overview
- What are queues?
- Queue purpose
- Queue vs list
- Queue benefits
- Queue Types

5. UVM Queues (2/5)

- ``uvm_queue`` - Generic queue
- Queue characteristics
- Queue operations
- Queue use cases
- Queue Usage
- Creating queues

5. UVM Queues (3/5)

- Adding items to queues
- Removing items from queues
- Queue management
- Queue Implementation
- Queue creation
- Queue operations

5. UVM Queues (4/5)

- Queue iteration
- Queue cleanup
- Queue Patterns
- Transaction queues
- Scoreboard queues
- Buffer queues

5. UVM Queues (5/5)

- Queue best practices

6. UVM String Utilities (1/3)

- String Utility Overview
- String manipulation needs
- UVM string utilities
- Utility benefits
- When to use utilities
- String Operations

6. UVM String Utilities (2/3)

- String formatting
- String conversion
- String manipulation
- String comparison
- String Utility Methods
- Common string operations

6. UVM String Utilities (3/3)

- Formatting functions
- Conversion functions
- Utility patterns

7. UVM Math Utilities (1/3)

- Math Utility Overview
- Mathematical operations
- UVM math utilities
- Utility benefits
- When to use utilities
- Math Operations

7. UVM Math Utilities (2/3)

- Random number generation
- Statistical functions
- Mathematical utilities
- Math patterns
- Math Utility Methods
- Common math operations

7. UVM Math Utilities (3/3)

- Random functions
- Statistical functions
- Utility patterns

8. UVM Random Utilities (1/3)

- Random Utility Overview
- Random number generation
- UVM random utilities
- Randomization support
- Random patterns
- Random Operations

8. UVM Random Utilities (2/3)

- Random value generation
- Constrained random
- Random seeds
- Random control
- Random Utility Methods
- Random number generation

8. UVM Random Utilities (3/3)

- Seed management
- Random state
- Utility patterns

9. UVM Primitives (1/3)

- Primitive Overview
- What are primitives?
- Primitive purpose
- Primitive types
- Primitive use cases
- Primitive Types

9. UVM Primitives (2/3)

- Common primitives
- Primitive operations
- Primitive characteristics
- Primitive selection
- Primitive Usage
- Using primitives

9. UVM Primitives (3/3)

- Primitive patterns
- Primitive best practices
- Primitive examples

10. UVM Macros (Python Context) (1/2)

- Macro Overview
- Macros in SystemVerilog vs Python
- Python alternatives
- Utility functions
- Helper decorators
- Python Equivalents

10. UVM Macros (Python Context) (2/2)

- Macro alternatives
- Utility functions
- Decorator patterns
- Helper classes

11. Utility Integration (1/3)

- Using Utilities in Testbenches
- When to use utilities
- Utility selection
- Utility integration
- Utility patterns
- Utility Best Practices

11. Utility Integration (2/3)

- Utility usage guidelines
- Performance considerations
- Memory management
- Utility organization
- Common Utility Patterns
- Command-line configuration

11. Utility Integration (3/3)

- Transaction comparison
- Transaction recording
- Object management

Hands-on examples

Module 8 self-check

```
$ ./scripts/module8.sh --check
```

Terminal: module8_check

```
#####  
Module 8: UVM Miscellaneous Utilities  
#####
```

Usage: ./scripts/module8.sh [OPTIONS]

Module 8: UVM Miscellaneous Utilities
This script runs examples and tests for Module 8.

OPTIONS:

Examples:

--clp Run Command Line Processor examples
--comparators Run Comparator examples
--recorders Run Recorder examples
--pools Run Pool examples
--queues Run Queue examples
--string-utils Run String utility examples
--math-utils Run Math utility examples
--random-utils Run Random utility examples
--integration Run Utility integration examples
--all-examples Run all examples (default)
--skip-examples Skip all examples

Tests:

--pyuvvm-tests Run pyuvvm tests

Environment:

--env DIR Virtual environment directory (default: .venv)
--no-venv Don't use virtual environment
--sim SIMULATOR Simulator to use (default: verilator)

Other:

--help, -h Show this help message

EXAMPLES:

Demo: Command Line Processor

```
$ .venv/bin/python module8/examples/clp/clp_example.py
```

Terminal: ex01_clp

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module8/examples/clp/clp_
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Comparators

```
$ .venv/bin/python module8/examples/comparators/comparator_example.py
```

Terminal: ex02_comparators

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module8/examples/comparat
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Recorders

```
$ .venv/bin/python module8/examples/recorders/recorder_example.py
```

Terminal: ex03_recorders

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module8/examples/recorder
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Pools

```
$ .venv/bin/python module8/examples/pools/pool_example.py
```

Terminal: ex04_pools

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module8/examples/pools/po
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Queues

```
$ .venv/bin/python module8/examples/queues/queue_example.py
```

Terminal: ex05_queues

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvim/module8/examples/queues/q
from pyuvim import *

ModuleNotFoundError: No module named 'pyuvim'

Demo: String Utilities

```
$ .venv/bin/python  
    module8/examples/string_utils/string_utils_example.py
```

Terminal: ex06_string_utils

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module8/examples/string_u
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Math Utilities

```
$ .venv/bin/python module8/examples/math_utils/math_utils_example.py
```

Terminal: ex07_math_utils

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module8/examples/math_utils_example.py", line 1, in
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Random Utilities

```
$ .venv/bin/python  
    module8/examples/random_utils/random_utils_example.py
```

Terminal: ex08_random_utils

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvvm/module8/examples/random_u
from pyuvvm import *

ModuleNotFoundError: No module named 'pyuvvm'

Demo: Utility Integration

```
$ .venv/bin/python module8/examples/integration/integration_example.py
```

Terminal: ex09_integration

Traceback (most recent call last):

File "/mnt/d/proj/universal-verification-methodology/learn_uvm_pyuvmm/module8/examples/integration/integration_example.py", line 1, in
from pyuvmm import *

ModuleNotFoundError: No module named 'pyuvmm'

Practice & assessment

Exercises

- Command Line Processor
- Comparator Implementation
- Recorder Usage
- Pool and Queue Usage
- Utility Integration

Assessment checklist

- Can use Command Line Processor
- Can implement and use comparators
- Can use recorders effectively
- Can use pools for object management
- Can use queues for data structures
- Can use string and math utilities

Summary & next steps

- Master UVM utility classes and helper functions
- Complete module8/CHECKLIST.md
- Run `./scripts/module8.sh --check`
- Next: Next module in course